CSE 3105 / CSE 3137

OBJECT-ORIENTED ANALYSIS AND DESIGN

FALL 2025

COURSE PROJECT: *<SMART CITY CLEAN>*

System Design Document

Group 5

ALEYNA GÜLTEKİN – 220315013

ESRA SEPİK – 220315015

ELİF SUDE YILMAZ – 220315022

ELİF ÖZKANAT – 220315023

NACİYE NUR AKKUŞ – 220315039

SİNEM TANRIVER – 220315073

22 December 2025

Table of Contents

1 Introduction 1
1.1 Purpose of the System 1
1.2 Design goals 1
2 Current Software Architecture 3
3 Proposed Software Architecture 4
3.1 Subsystem decomposition 6
3.2 Hardware/software mapping 13
3.3 Persistent data management 15
3.4 Access control and security 16
3.5 Boundary conditions 18
4 Subsystem Services 18
5 Glossary 19
6 References 20
7 Appendix 20

List of Figures

Figure 1: Identifying Design Trade-offs. 2
Figure 2: Data Management Subsytem – Class Diagram 6
Figure 3: Waste Management Subsytem - Class Diagram 6
Figure 4: Route Optimization Subsytem - Class Diagram 7
Figure 5: Task & Notification Subsytem – Class Diagram 7
Figure 6: IoT / Smart Bin Subsytem (Class Diagram) 8
Figure 7: Subsytem Decomposition (Unified Class Diagram) 9
Figure 8: Subsytem Decomposition , Package Diagram (Component – High Level Architecture) 11
Figure 9: Software Architecture (UML Component Diagram) 13
Figure 10: Hardware Software Mapping (Deployment Diagram) 14
Figure 11: Access Matrix 17
Figure 12: System Boundry 18
Figure 13: Subsystem Services , Interface/Lollipop Notation 19

1 Introduction

This System Design Document (SDD) provides a comprehensive overview of the software architecture and design decisions for the **Smart City Clean – Smart Waste and Cleaning Management System**. The primary objective is to transform the functional and nonfunctional requirements identified in the Requirements Analysis Document (RAD) into a technical solution. This document describes the system decomposition into subsystems, their hardware/software mapping, data management strategies, and the services provided by each component.

1.1 Purpose of the System

The Smart City Clean system is an intelligent management application designed to eliminate urban waste issues like overflowing bins and inefficient collection routes. By integrating IoT sensors with a centralized processing unit, the system provides real-time monitoring of waste levels, triggers alerts for critical fullness (FR2.0), and generates optimized collection routes (UC-R01) for cleaning staff. The system serves three main user groups: Citizens (for reporting), Cleaning Staff (for task execution), and Municipal Managers (for monitoring and performance analysis).

1.2 Design goals

Design goals define the high-level characteristics of the **Smart City Clean** system. They are derived from the nonfunctional requirements and serve as a guide for making architectural trade-offs during the system design phase.

Technical Goals

- **Performance (High Priority):** Based on **FR3.0**, the system must generate optimized routes in less than 5 seconds. To achieve this, the computational load of the Route Optimization Algorithm must be prioritized, even if it requires more server-side resources.
- **Reliability (High Priority):** The system must ensure a 99% success rate for alert delivery (FR2.0) and sensor data synchronization. Any failure in notifying the staff about overflowing bins directly impacts urban hygiene.
- **Scalability (Medium Priority):** As the city grows, the system should support an increasing number of smart bins (IoT nodes) and concurrent mobile users without a significant drop in performance.

User-Centric Goals

- **Usability (Medium Priority):** The staff application must be simple enough to operate in field conditions. The citizen application must allow reporting with minimal steps (e.g., photo upload and auto-location) to encourage public participation.
- **Security (High Priority):** As required in Section 3.2, all communications between IoT sensors, mobile apps, and the server must be encrypted via TLS/SSL. Access to the Municipal Dashboard is strictly controlled via an access matrix (AC1).

Identifying Design Trade-offs

Trade-off	Rationale
Performance (Speed) vs. Optimality (Accuracy)	To meet the 5-second route generation limit, the system uses heuristic algorithms. We prioritize speed over finding the "perfect" mathematical shortest path to ensure real-time responsiveness. (Selected Priority: Performance)
Security vs. Usability (Ease of Access)	Strong encryption and multi-factor authentication for managers increase security but slightly add complexity to the login process. We prioritize security for administrative tasks to protect city data. (Selected Priority: Security)
Real-time Monitoring vs. Battery Life (IoT Cost)	Continuous sensor data transmission provides perfect real-time data but drains batteries quickly. We use Event-Driven transmission (only sending data when a threshold is hit) to balance monitoring with energy efficiency. (Selected Priority: Efficiency)

Figure 1: Identifying Design Trade-offs.

During the architectural design of the system, critical decision points (trade-offs) were identified to balance conflicting design goals. These choices were analyzed in line with the project's sustainability and user needs as follows:

1. Performance (Speed) vs. Optimization (Mathematical Accuracy)

Choice: Heuristic Algorithms were preferred over Exact Optimization (e.g., Brute-force TSP) Algorithms.

Rationale: Finding the absolute shortest path for hundreds of waste bins city-wide is an NP-hard problem, and processing time increases exponentially as the number of bins grows. To satisfy the design goal of **"Generating a route in less than 5 seconds" (FR3.0)**, we selected heuristic approaches that provide a "near-optimal" result within seconds rather than a perfect solution that could take minutes or hours.

2. Energy Efficiency (Battery Life) vs. Real-Time Data Flow

Choice: Event-Driven Data Transmission was preferred over Continuous Polling.

Rationale: Continuous data transmission from IoT sensors would provide perfect real-time monitoring but would drain the batteries of thousands of field devices very quickly, leading to high maintenance costs. Therefore, by selecting a model that transmits data only when a threshold (e.g., 80% fullness) is exceeded or at specific daily intervals, we optimized hardware longevity and operational efficiency.

3. Security vs. Usability (Ease of Access)

Choice: Multi-Factor Authentication (MFA) and TLS 1.3 Encryption were preferred over Anonymous/Fast Access.

Rationale: The municipal dashboard contains critical city data and sensitive personnel information. While fast login enhances user experience, preventing data breaches is our primary concern. Prioritizing security protocols (MFA and SSL) ensures the **Reliability** and **Integrity** of the system, protecting municipal infrastructure from unauthorized access.

4. Data Consistency vs. Write Speed

Choice: Relational Database (PostgreSQL/RDBMS) was preferred over certain NoSQL databases.

Rationale: There is a strong relationship between waste collection reports, personnel assignments, and bin status. Maintaining strict data consistency (ACID properties) was deemed more important than achieving ultra-high write speeds. This choice ensures that municipal reporting remains error-free and transactionally sound.

2 Current Software Architecture

During the design phase of the "Smart City Clean - Smart Waste and Cleaning Management System" project, the architectural approaches of industry-recognized systems were examined. The strengths and weaknesses of these systems were evaluated in alignment with the "High Coherence" and "Low Coupling" objectives of our proposed architecture.

2.1. Enevo: Cloud-Native Dynamic Routing Architecture

Enevo is a pioneering SaaS (Software as a Service) platform in the waste management sector that integrates IoT sensors with cloud-based analytics. The core of the system relies on a dynamic routing engine operating on a centralized cloud server.

Architectural Approach: The system utilizes a fully cloud-native structure centered around REST APIs.

Strengths:

High Scalability: The cloud infrastructure allows for the simultaneous processing of data from thousands of containers.

Dynamic Updates: Route optimization algorithms are updated centrally, providing instantaneous reflections to field units.

Weaknesses:

Dependency: The system's decision-making capability completely ceases in the event of an internet connectivity loss (High External Dependency).

Data Privacy: All sensitive urban data is stored on third-party cloud provider infrastructures.

2.2. Bigbelly: Integrated and Closed Eco-system Architecture

Bigbelly manages its proprietary smart hardware, such as solar-powered compactors, through a software architecture known as the "CLEAN Management Console".

Architectural Approach: This is a vertical integration model where hardware and software are tightly interconnected (Highly Coupled).

Strengths:

Autonomous Decision Making: Waste bins can perform operations like waste compaction independently of the central system (Edge Computing).

High Reliability: Since hardware and software are developed by the same manufacturer, compatibility issues are minimized.

Weaknesses:

Low Modularity: It is nearly impossible to integrate sensors from different brands or customized algorithms into the system (Vendor Lock-in).

Maintenance Difficulty: A change in a single component can impact the entire integrated structure.

3 Proposed Software Architecture

The proposed software architecture for the Smart City Clean system is designed as an integrated, data-driven platform that transitions urban waste management from static, inefficient schedules to a dynamic, real-time optimization model. The architecture is structured to ensure high system reliability and performance by separating client-side interactions from complex server-side processing.

Architectural Overview and Value Proposition:

Our proposal leverages a multi-tiered server-side architecture where core functionalities are encapsulated into specialized subsystems to maintain High Coherence. The system's primary value lies in its ability to process real-time Internet of Things (IoT) data and transform it into actionable intelligence for municipal operations.

Dynamic Optimization Engine: Unlike traditional systems with fixed schedules, our architecture proposes an Optimization Subsystem that calculates the most efficient collection routes based on live fill-level data and citizen reports in less than 5 seconds.

IoT-Driven Monitoring: The Sensor Management Subsystem acts as the system's perception layer, continuously fetching compartment fill levels (Metal, Paper, Plastic, Glass, Household Waste) to ensure no bin exceeds its capacity unnoticed.

Proactive Alerting: The Notification Subsystem ensures high operational reliability (99%) by automatically dispatching urgent alerts to cleaning personnel when bins exceed an 80% fullness threshold.

Subsystem Rationale and Design Goals

The decomposition of the system into distinct modules aims to achieve Low Coupling, ensuring that modifications to one component—such as updating the optimization algorithm—do not necessitate changes in the data access or notification logic.

Data Encapsulation: To enhance security and maintainability, direct database access is strictly prohibited. We propose a Database Access Subsystem that serves as an abstraction layer, providing controlled data interfaces to all other functional modules.

Multi-Actor Interface Strategy: The architecture provides tailored interfaces for three distinct actors:

Citizens: Facilitating active participation through pollution and overflow reporting via the Mobile Client.

Cleaning Staff: Receiving real-time task updates and optimized navigation through specialized mobile applications.

Municipal Managers: Gaining a holistic view of city hygiene and system performance through a responsive Web Dashboard.

By centralizing core system logic within a secure application server while distributing interaction points through mobile and web clients, the proposed architecture satisfies the critical non-functional requirements of security, scalability, and supportability.

3.1 Subsystem decomposition

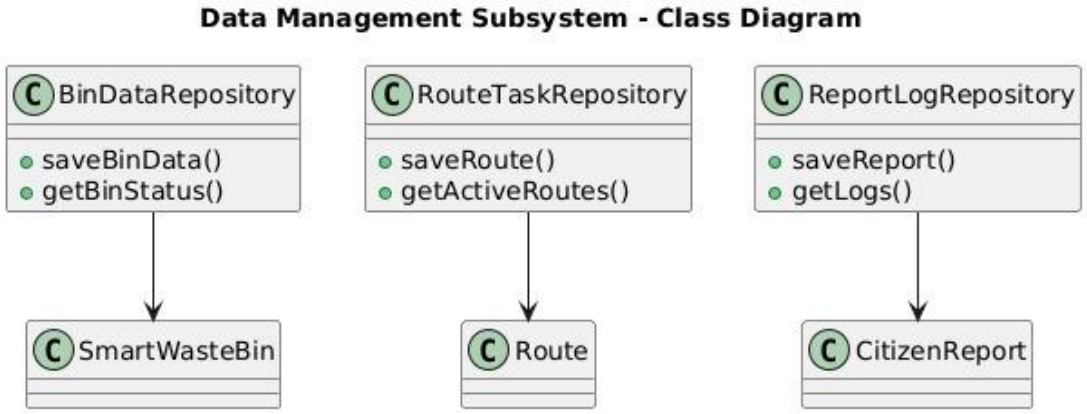


Figure 2: Data Management Subsytem – Class Diagram

The Data Management Subsystem serves as the digital memory of the project, ensuring that all operational records are securely stored and retrieved whenever needed. The repositories within this system maintain real-time fill-level data from smart waste bins, citizen-submitted reports, and historical records of generated routes. By organizing the raw data required by all other components, this layer provides a consistent and reliable foundation for the entire system's operations.

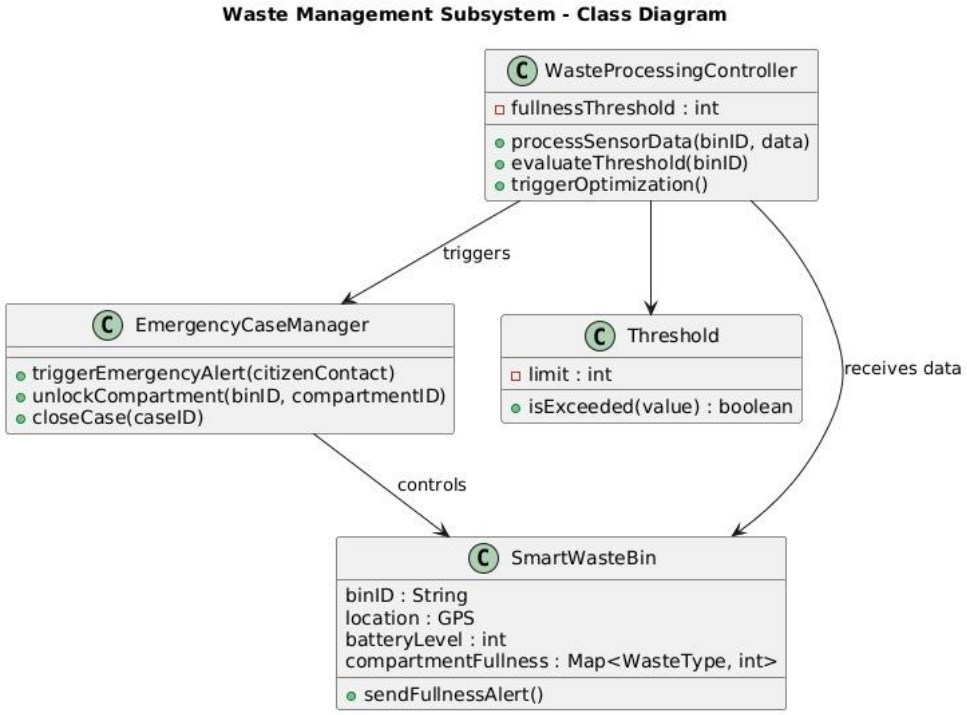


Figure 3: Waste Management Subsytem - Class Diagram

The Waste Management Subsystem acts as the central control mechanism that monitors the physical status of smart bins in the field and interprets incoming data. Processing controllers analyze sensor data—such as battery levels and compartment fullness—against predefined threshold values. If a bin exceeds its capacity limit or if an unexpected fault occurs, an

emergency manager steps in to trigger alerts or initiate specific interventions, such as unlocking compartments for maintenance.

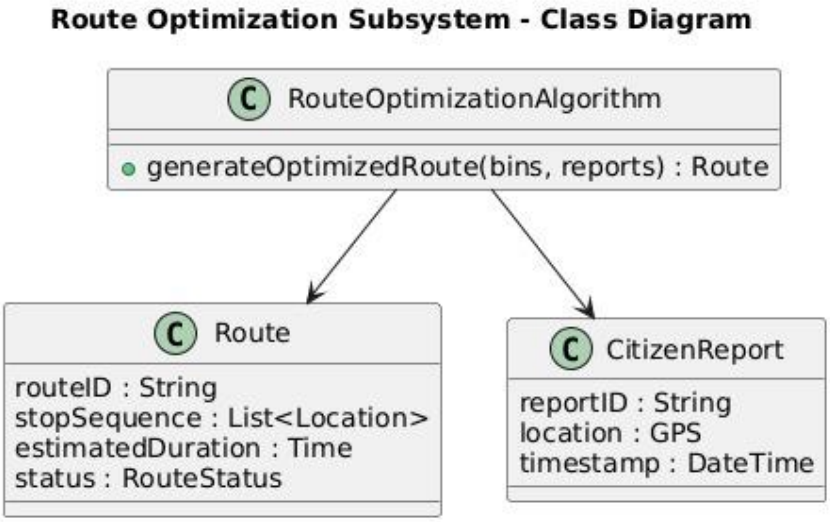


Figure 4: Route Optimization Subsytem - Class Diagram

The Route Optimization Subsystem maximizes operational efficiency by utilizing fill-level data from the waste management unit and feedback from citizen reports. Advanced optimization algorithms analyze all points requiring collection to generate the most efficient route plan, ensuring significant fuel savings and better time management. In this process, the system calculates detailed stop sequences and estimated durations by considering not only full bins but also urgent reports submitted by the public.

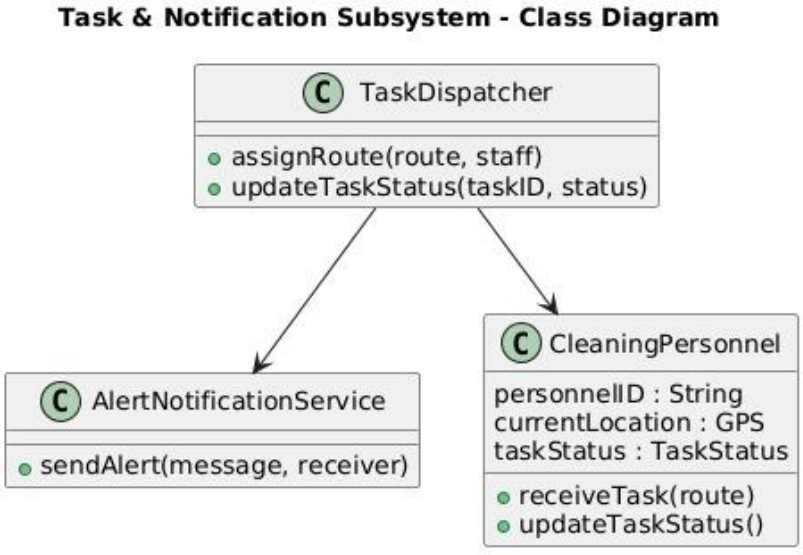


Figure 5: Task & Notification Subsytem – Class Diagram

The Task and Notification Subsystem is the operational bridge that transforms digital plans into real-world actions. A task dispatcher manages the workflow by assigning optimized routes to the appropriate cleaning personnel in the field. Once a task is assigned, the notification service immediately informs the staff, allowing the system to track their location

and task status in real-time. In this way, the system completes the full cycle from initial data collection to the successful completion of field tasks.

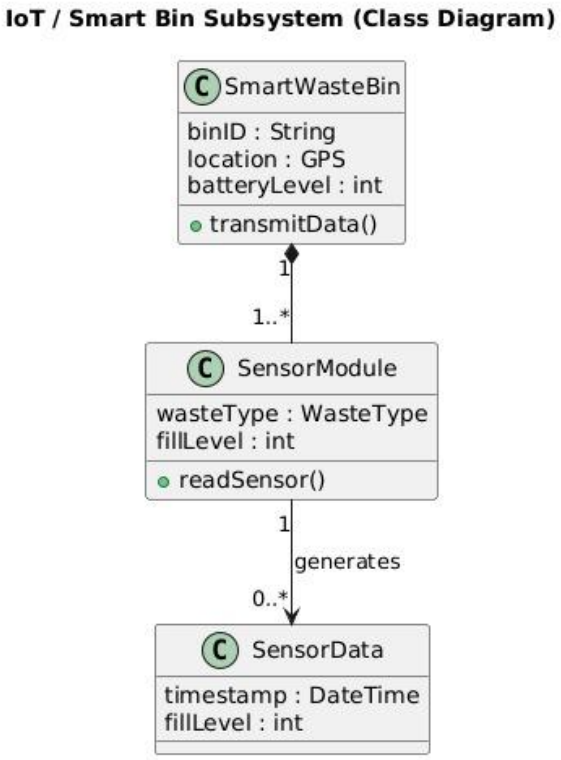


Figure 6: IoT / Smart Bin Subsytem (Class Diagram)

This class diagram represents the structural design of an IoT Smart Waste Bin system. At the core is the SmartWasteBin class, which manages the bin's unique ID, GPS location, and battery status, while using the transmitData() method to send information to a central server. This bin is composed of one or more SensorModules that monitor specific waste types and fill levels through the readSensor() function. Each sensor module generates SensorData records, which log precise fill-level measurements along with a timestamp for historical tracking. Essentially, the system defines a hierarchy where a physical bin uses integrated sensors to collect and report real-time waste data.

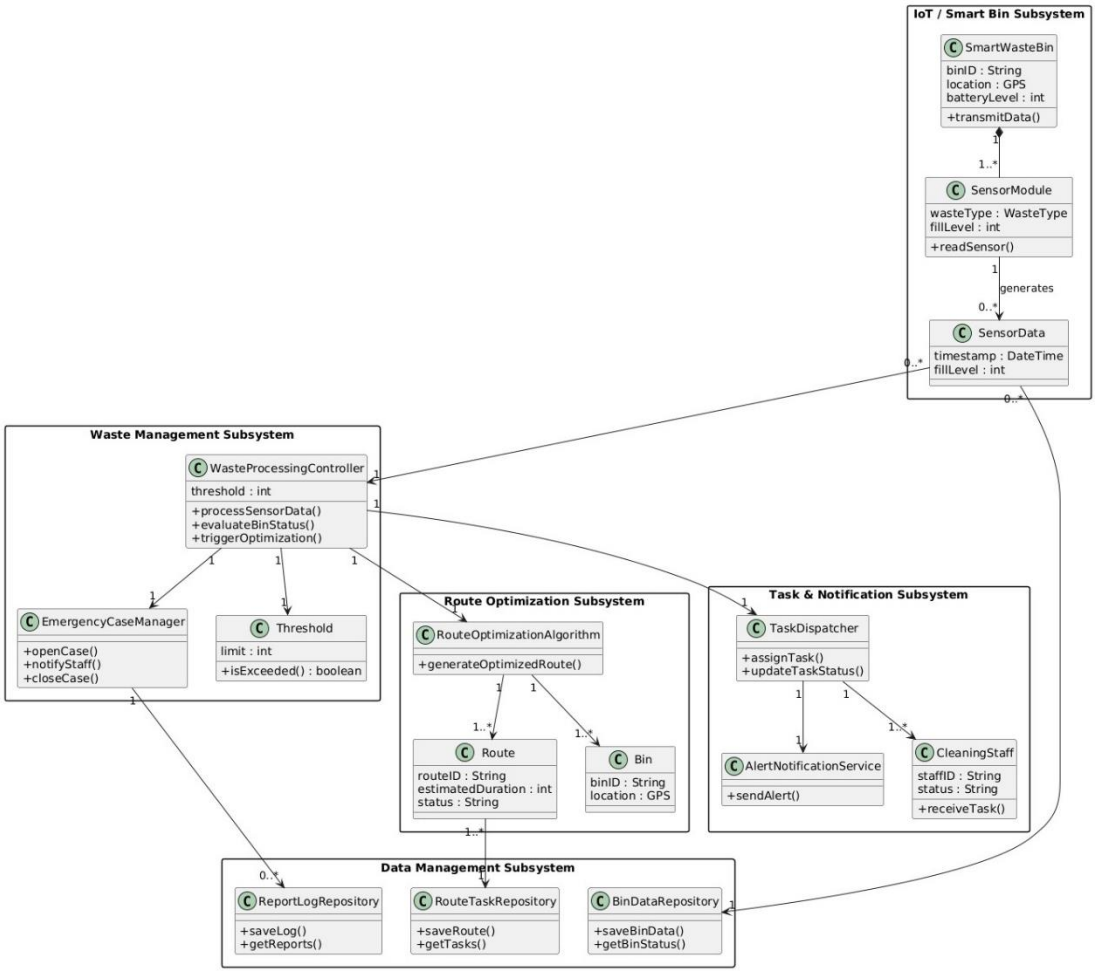


Figure 7: Subsytem Decomposition (Unified Class Diagram)

Subsystem Decomposition Based on the CRUD Matrix

The provided CRUD matrix outlines the access rights and functional responsibilities within the system. Based on the interactions between actors and system objects, the architecture is decomposed into three main functional subsystems:

1. Public Reporting Subsystem (Citizen Module)

This subsystem manages the interaction between the general public and the municipality.

Primary Actor: Citizen.

Key Responsibilities: It allows users to initiate the workflow by creating CitizenReport objects.

Data Access: It provides transparency by allowing citizens to Read SmartBinData (e.g., checking if a bin is full), but restricts any modification to the infrastructure or logistics data.

2. Field Operations Subsystem (Staff Module)

This module is designed for the execution and monitoring of waste management tasks in the field.

Primary Actor: Cleaning Staff.

Key Responsibilities: Staff members interact with CleaningRoute and StaffTask objects. While they can only Read their assigned routes, they have the authority to Update their task status.

Accountability: The staff is responsible for generating SystemLog entries (Create permission) to document operational progress or issues encountered during their shift.

3. Administrative & Governance Subsystem (Manager Module)

This is the central control unit of the system, possessing the highest level of authority.

Primary Actor: Municipal Manager.

Key Responsibilities: The manager has full lifecycle control (Create, Read, Update, Delete) over critical business entities such as CleaningRoute, StaffTask, and UserAccount.

System Integrity: This subsystem is the only one capable of performing Delete operations on reports and accounts, ensuring that data cleanup and user management are handled exclusively by authorized personnel.

Integration into the Unified Class Diagram

In the Unified Class Diagram, this matrix dictates the Public Methods and Access Modifiers for each class:

Methods: The CRUD operations translate directly into class methods. For example, the StaffTask class will include updateStatus() for the Staff and createTask()/deleteTask() for the Manager.

Visibility: Classes like UserAccount will have restricted associations, visible only to the Administrative Subsystem.

Relationships: The diagram will show a Dependency relationship between the Cleaning Staff and the SystemLog, and a Composition relationship between the Municipal Manager and UserAccount management.

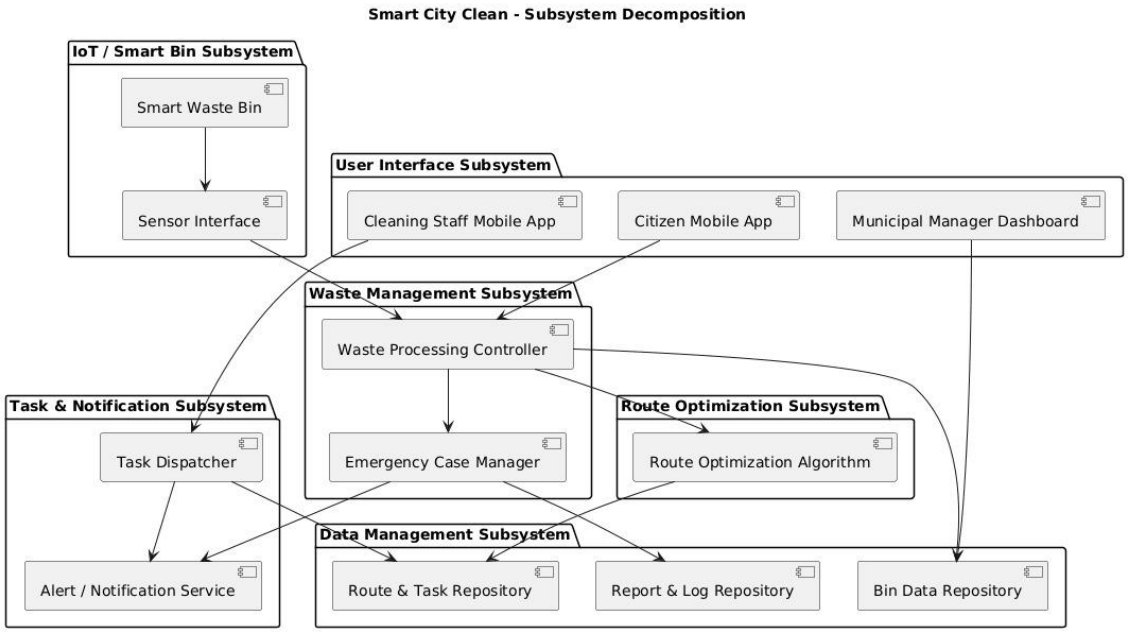


Figure 8: Subsytem Decomposition , Package Diagram (Component – High Level Architecture)

This section provides a comprehensive breakdown of the six main subsystems designed for the Smart City Clean project, detailing their internal components and the communication flows between them.

1. IoT / Smart Bin Subsystem

Components: Includes the physical Smart Waste Bin and the Sensor Interface for network communication.

Responsibility: Collects real-time fill level data from compartments (FR1.0) and manages waste classification (FR7.0).

Relationships: The Sensor Interface pushes raw sensor data directly to the Waste Management Subsystem for processing.

2. User Interface Subsystem

Components: Comprises the Cleaning Staff Mobile App, Citizen Mobile App, and Municipal Manager Dashboard.

Responsibility: Provides the entry point for all actors. It facilitates citizen reporting (FR6.0), staff task tracking (FR4.0), and managerial monitoring (FR5.0).

Relationships: Mobile clients send requests to the Waste Management Subsystem, while the Manager Dashboard fetches data directly from the Data Management Subsystem to visualize bin statuses.

3. Waste Management Subsystem (Central Controller)

Components: Houses the Waste Processing Controller (the central brain) and the Emergency Case Manager.

Responsibility: Orchestrates business logic and coordinates complex scenarios, such as the valuable item retrieval process (FR8.0).

Relationships: Acts as the central hub. It receives IoT data, triggers the Route Optimization Subsystem, and directs the Task & Notification Subsystem to alert personnel.

4. Route Optimization Subsystem

Components: An isolated layer containing the Route Optimization Algorithm.

Responsibility: Performs mathematical processing to generate the most efficient collection routes based on live data and reports (FR3.0).

Relationships: Operates upon requests from the Waste Management Subsystem and stores the generated routes in the Data Management Subsystem.

5. Task & Notification Subsystem

Components: Consists of the Task Dispatcher and the Alert / Notification Service.

Responsibility: Handles the assignment of routes to staff and manages the dispatch of urgent alerts (>80% fullness, FR2.0).

Relationships: Triggered by Waste Management, it updates the Data Management repositories and communicates with field staff via the notification service.

6. Data Management Subsystem (Persistence Layer)

Components: Includes Route & Task, Report & Log, and Bin Data repositories.

Responsibility: Encapsulates direct database access to ensure data integrity and maintains historical records/system logs (FR9.0).

Relationships: Serves as the foundational layer that all other subsystems interact with to store or retrieve persistent information.

3.2 Hardware/software mapping

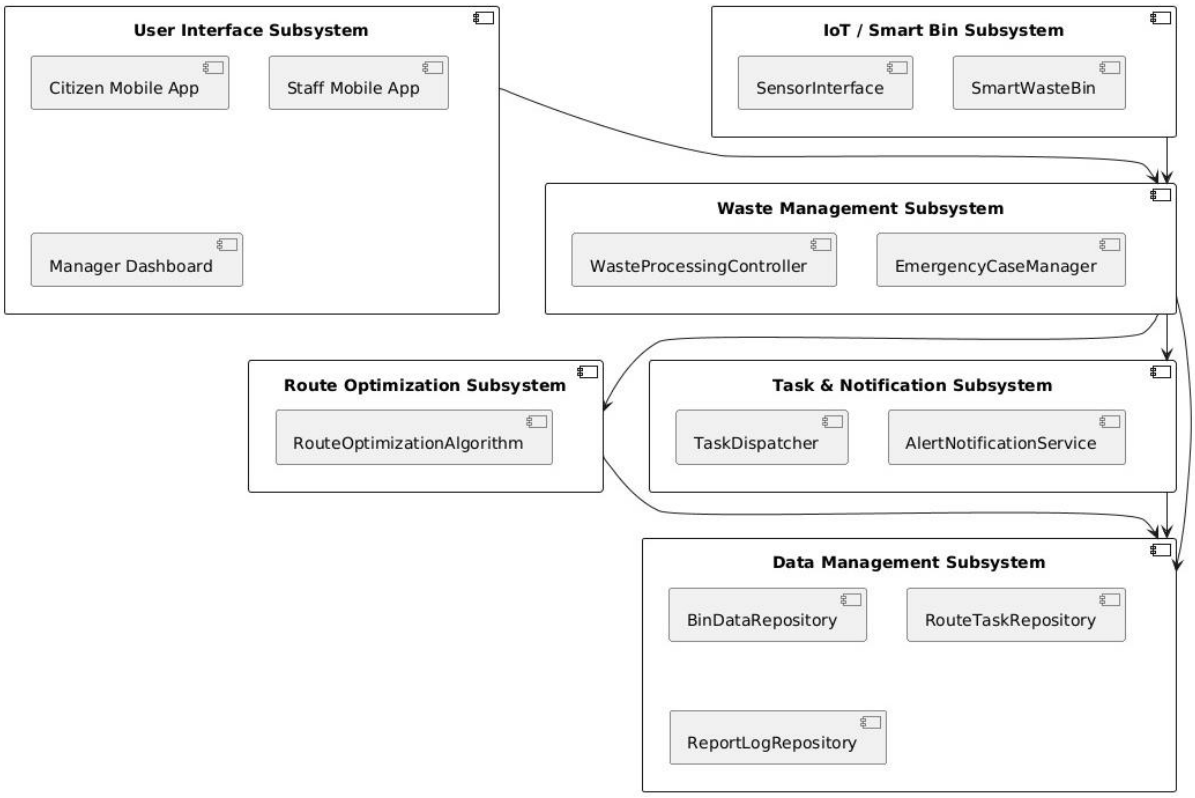


Figure 9: Software Architecture (UML Component Diagram)

The hardware/software mapping section describes the physical distribution of the previously defined subsystems across hardware nodes and execution environments. Our architecture follows a 3-Tier Distributed Client-Server pattern to ensure scalability and high availability.

Architectural Mapping and External Components

SmartWasteBin (IoT Device): This node hosts the IoT / Smart Bin Subsystem. It utilizes a low-power microcontroller (e.g., ESP32) as the BinController and interfaces with ultrasonic and weight sensors via the SensorModule.

Client Devices (Mobile/Web): The User Interface Subsystem is distributed across mobile devices (for Staff and Citizens) and web browsers (for Managers). These clients communicate with the server via HTTPS/REST protocols using JSON for data exchange.

Application Server (Cloud/Municipality Data Center): This is the core of the system. It hosts the System Core execution environment, including:

Waste Management & Task Dispatcher: Managing business logic.

Route Optimization Engine: Utilizing external libraries (e.g., Google Maps OR-Tools or OpenRouteService) to calculate routes in < 5 seconds.

Database Server: A dedicated environment hosting a PostgreSQL relational database. This separation ensures that the Data Management Subsystem remains independent of the application logic, allowing for secure and persistent storage of bin fill levels and system logs.

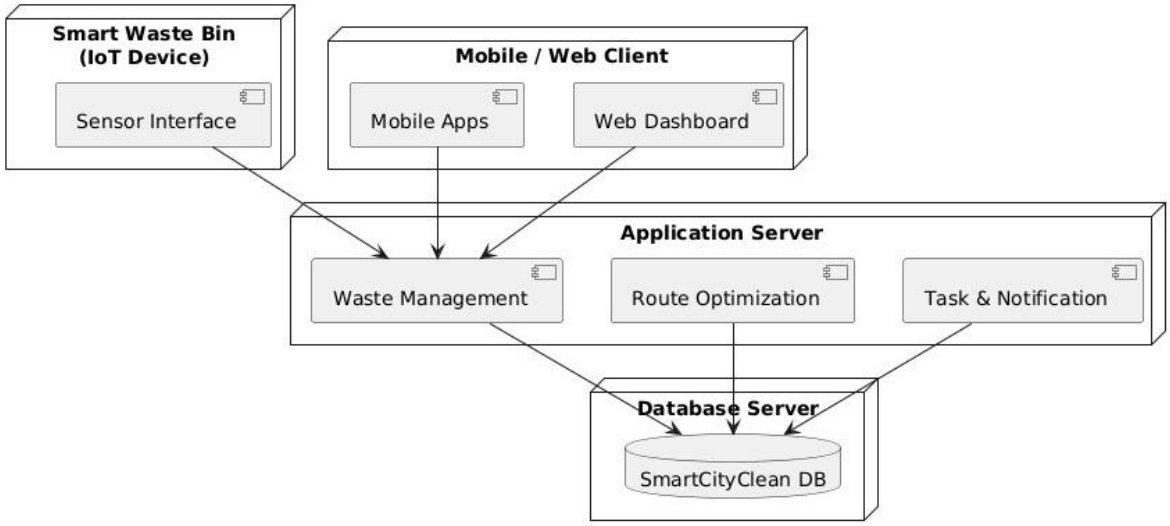


Figure 10: Hardware Software Mapping (Deployment Diagram)

Hardware/Software Mapping and Subsystem Distribution

The hardware/software mapping section describes the physical distribution of the previously defined subsystems across hardware nodes and execution environments. Our architecture follows a 3-Tier Distributed Client-Server pattern to ensure scalability and high availability.

SmartWasteBin (IoT Device): This edge node hosts the IoT / Smart Bin Subsystem. It utilizes a low-power microcontroller as the BinController and interfaces with ultrasonic and weight sensors via the Sensor Interface to monitor waste levels in real-time.

Mobile / Web Client: This layer hosts the User Interface Subsystem, which is distributed across Mobile Apps (for Cleaning Staff and Citizens) and a Web Dashboard (for Municipal Managers). These clients facilitate interaction with the system and communicate with the central server via secure HTTPS/REST protocols using JSON for data exchange.

Application Server: Located in a Cloud or Municipality Data Center, this node serves as the System Core execution environment. It integrates the Waste Management Subsystem (comprising the Waste Processing Controller and Emergency Case Manager), the Route Optimization Subsystem (running the Route Optimization Algorithm), and the Task & Notification Subsystem (handling the Task Dispatcher and Alert/Notification Service).

Database Server: This tier provides a dedicated environment for the Data Management Subsystem, hosting the SmartCityClean DB. This ensures that the Bin Data, Route & Task, and Report & Log Repositories remain independent of the application logic, providing secure and persistent storage for all system records.

3.3 Persistent data management

The Smart City Clean – Smart Waste and Cleaning Management System requires persistent data storage to support real-time monitoring, route optimization, reporting, and system analysis. Persistent data refers to the information that must survive system shutdowns and be available across multiple executions of the system.

Persistent Data Description

Based on the Requirements Analysis Document, the system stores the following persistent data:

-Smart Waste Bin Data:

Bin identifiers, geographical locations, battery levels, and compartment fill levels (Metal, Paper, Plastic, Glass, Household Waste), along with timestamped sensor readings.

-Citizen Reports:

Pollution and overflowing bin reports submitted by citizens, including location information, photos, timestamps, and report status.

-Route and Task Data:

Optimized route information generated by the system, including route identifiers, stop sequences, estimated durations, and task completion statuses reported by cleaning staff.

-User and Actor Data:

Information related to system actors such as Cleaning Staff and Municipal Managers, including role definitions and access privileges.

-System Logs and Historical Records:

Records of alerts, incorrect waste disposal attempts, emergency item retrieval events, and task completion logs for performance analysis and reporting purposes.

Data Management Infrastructure

Due to the concurrent access requirements of the system—where multiple smart bins, mobile applications, and web dashboards interact with the data simultaneously—a relational database management system (RDBMS) is selected. A relational database provides consistency, reliability, and support for concurrent read/write operations, which are essential for real-time urban management systems.

The object model defined in the analysis phase is mapped to the relational schema as follows:

-Each persistent class is mapped to a database table.

-Class attributes are represented as table columns.

-Associations between classes are implemented using foreign keys.

-Timestamped records are used to support historical analysis and reporting.

Database Encapsulation

To ensure maintainability, security, and low coupling between subsystems, direct access to the database is not allowed. Instead, the database is encapsulated within a Data Management Subsystem, which provides controlled data access through well-defined interfaces. All other subsystems, including the Central System, Route Optimization Subsystem, Mobile Application, and Dashboard, interact with the persistent data exclusively through this subsystem.

This encapsulation approach improves data integrity, supports future database technology changes, and aligns with the system's design goals of maintainability, security, and scalability.

3.4 Access control and security

The Smart City Clean – Smart Waste and Cleaning Management System enforces a multi-layered security strategy to protect system functionalities and data integrity. This approach integrates role-based access control, secure authentication, and robust encryption to satisfy both functional and non-functional security requirements.

Role-Based Access Control and User Model The system distinguishes between three primary actors—Citizen, Cleaning Staff, and Municipal Manager—restricting their actions according to their specific responsibilities defined in the Requirements Analysis Document. Access rights are formalized through a role-based model which ensures that each user is granted only the permissions necessary for their specific tasks. This structure prevents unauthorized operations while providing managers with full visibility and administrative control over the system.

User Model and Access Matrix

The system defines three primary actors: Citizen, Cleaning Staff, and Municipal Manager. Each actor is granted access only to the functionalities necessary to perform their tasks. Table summarizes the access rights of each actor in the form of an access matrix.

Actor \ System Object	CitizenReport	SmartBinData	CleaningRoute	StaffTask	SystemLog	UserAccount
Citizen	Create	Read	-	-	-	-
Cleaning Staff	Read	Read	Read	Update	Create	-
Municipal Manager	Read, Delete	Read, Update	Create, Read, Update, Delete	Create, Read, Update, Delete	Read	Create, Read, Update, Delete

Figure 11: Access Matrix

Authentication and Session Management Access to protected resources is guarded by a centralized authentication mechanism managed by the Central System Subsystem. While Municipal Managers and Cleaning Staff must provide secure credentials through web or mobile interfaces, Citizens can submit waste reports without authentication to maintain system accessibility. However, all anonymous submissions undergo validation to mitigate potential misuse.

Data Protection and Encryption Architecture To ensure the confidentiality of sensitive information, the system utilizes secure communication protocols for all data transmission between smart bins, mobile apps, web dashboards, and the central server. This protection extends to data at rest, where sensitive records like user credentials and system logs are secured through access control mechanisms and secure storage practices.

Key Management and Security Maintenance Consistency and security are maintained through centralized key management, where encryption keys and credentials are kept isolated from client applications. The system implements regular key rotation and secure storage practices to minimize the risk of compromise and ensure long-term operational security.

3.5 Boundary conditions

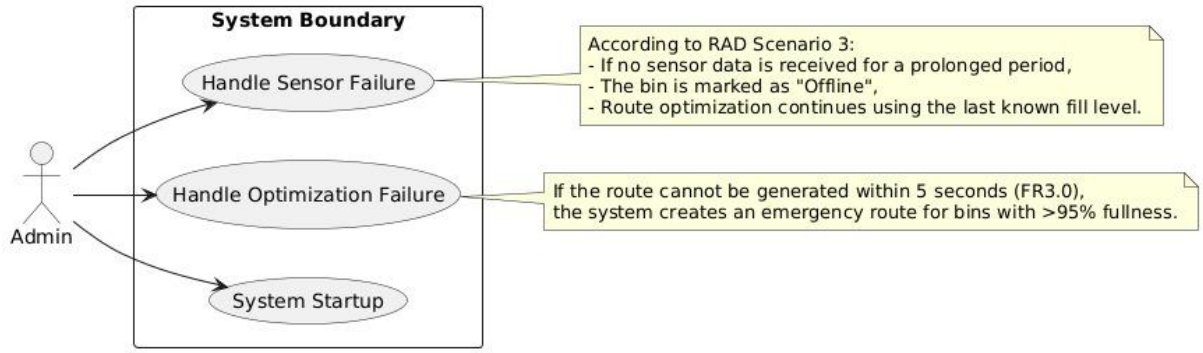


Figure 12: System Boundry

The system's operational integrity is maintained through clearly defined boundary conditions that manage startup, error handling, and performance limits. These measures ensure that the system remains safe and functional even under unexpected conditions. The specific execution logic for these states is detailed below:

1. System Startup (Initialization) This use case represents the primary entry point for system operations. Upon startup, the software initializes by loading configuration parameters, establishing secure communication with IoT sensors, and preparing the route optimization module for normal execution.

2. Handle Sensor Failure (Resilience) Triggered when no sensor data is received for a prolonged period, this process follows RAD Scenario 3 to ensure service continuity:

The affected smart bin is automatically marked as “Offline” within the system.

The optimization module continues to calculate routes using the bin's last known fill level, preventing a total system halt due to localized hardware issues.

3. Handle Optimization Failure (Critical Fallback) This fallback mechanism is activated if the system fails to generate an optimized route within the required 5-second threshold (FR3.0):

The system bypasses standard optimization to automatically create an emergency route.

This emergency route prioritizes bins with a fullness level exceeding 95% to ensure that the most critical areas are serviced immediately.

4 Subsystem Services

The architecture of the system is built upon modular subsystem services that interact through well-defined interfaces to ensure low coupling and high maintainability. At the core of data management, the I_DataAccess interface provides a secure gateway for subsystems to store task histories and log sensor values without having direct access to the database. Real-time communication is facilitated by the I_SensorData interface, which allows the Core Management subsystem to receive bin updates from the IoT Gateway. User-facing operations are managed through the I_RouteService and I_TaskService interfaces; these allow the Manager Dashboard to monitor active routes and the Staff

Mobile App to receive and execute specific cleaning tasks. By encapsulating complex logic like the RouteOptimizationEngine behind these services, the system allows for independent updates to individual components without disrupting the overall service flow.

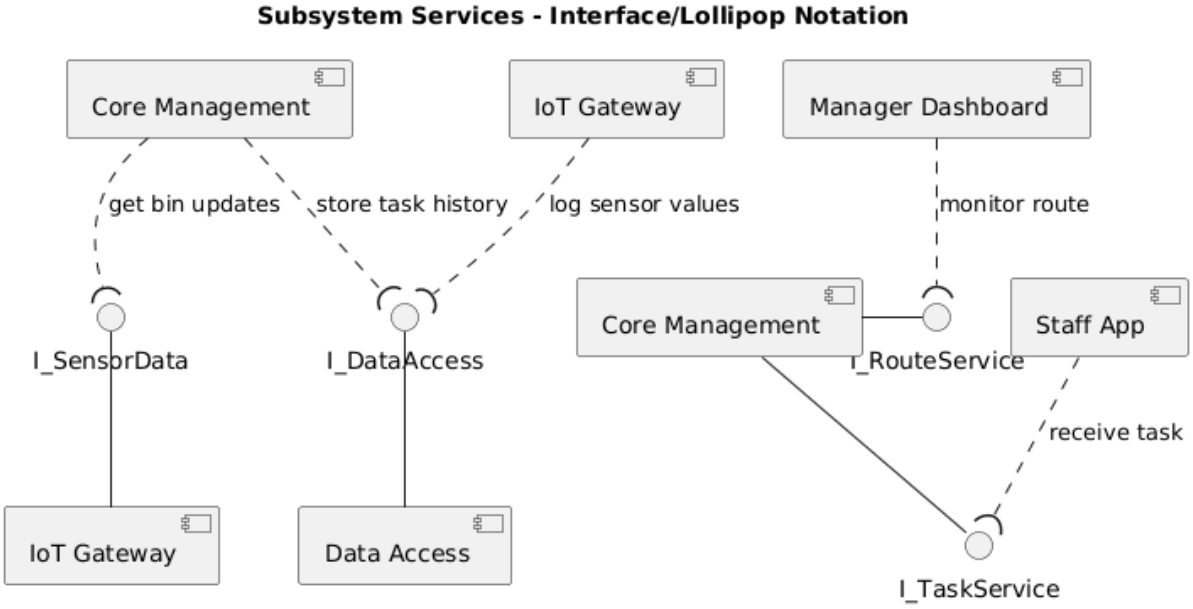


Figure 13: Subsystem Services , Interface/Lollipop Notation

5 Glossary

Access Matrix: A table defining which system operations are permitted for each actor role.

Central System / System Core: The server-side component responsible for coordinating business logic and subsystem interactions.

Citizen: An actor who reports pollution or overflowing bins through the mobile application.

Cleaning Staff: An actor responsible for executing assigned waste collection tasks using the staff mobile application.

Data Management Subsystem: The subsystem that encapsulates database access and manages persistent system data.

FR (Functional Requirement): A requirement that specifies a function the system must perform, as defined in the RAD.

IoT (Internet of Things): A network of physical devices equipped with sensors that collect and transmit data.

IoT / Smart Bin Subsystem: The subsystem responsible for collecting and transmitting real-time waste data from smart bins.

Municipal Manager: An actor who monitors system performance and oversees waste management operations.

Route Optimization Algorithm: The component that generates optimized waste collection routes based on real-time data.

Smart City Clean: The Smart Waste and Cleaning Management System described in this document.

Subsystem: A logical grouping of related components providing a specific functionality.

Subsystem Decomposition: The process of dividing the system into manageable subsystems.

Task Dispatcher: A component that assigns optimized routes and tasks to cleaning staff.

UC (Use Case): A description of interactions between an actor and the system to achieve a goal.

6 References

https://enevo.com/

https://bigbelly.com/products/clean-software

7 Appendix

- Annex – I: Distribution of Work
- Annex – II: Meeting Minutes

Distribution of Work

The workload for the System Design Document (SDD) was strategically distributed among the six team members to ensure a well-balanced and technically comprehensive contribution.

Aleyna and **Elif** focused on the technical modeling of the system; while Aleyna developed the architectural decomposition and hardware/software mapping diagrams, Elif designed the detailed subsystem service interfaces using lollipop notation. **Sinem** and **Esra** were responsible for the technical documentation and rationale of these models, ensuring that every architectural decision was justified based on design goals. **Elif Sude** conducted a thorough analysis of the system's nonfunctional requirements to define high-level design goals and led the discussion on critical design trade-offs. Finally, **Naciye Nur** developed the access control matrix and security protocols, while also managing the professional formatting and final consolidation of the SDD into a formal report. This collaborative effort ensured that both the solution domain and the documentation standards were fully addressed.

Annex – II

Meeting Minutes

Date:	08.12.2025
Location:	Engineering Faculty
Duration:	3 hours
Participants:	Aleyna Gültekin, Esra Sepik, Elif Sude Yılmaz, Elif Özkanat, Naciye Nur Akkuş, Sinem Tanrıver

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

The primary agenda of the first meeting was to review the feedback from the RAD report and establish the architectural foundations for the System Design Document (SDD). The team decided to adopt a **Client-Server architecture** based on functional partitions to handle the real-time requirements of the Smart City Clean project. **Elif Sude** led the discussion on prioritizing design goals, focusing on Performance (FR3.0) and Reliability. **Sinem** and **Esra** began defining the initial boundaries of the subsystems (IoT, Routing, and User Management). Preliminary trade-off decisions were discussed, specifically balancing the speed of route optimization against mathematical optimality.

Annex – II

Date:	15.12.2025
Location:	Engineering Faculty
Duration:	3 hours
Participants:	Aleyna Gültekin, Esra Sepik, Elif Sude Yılmaz, Elif Özkanat, Naciye Nur Akkuş, Sinem Tanrıver

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

The second meeting focused on the formalization of UML models and technical documentation. **Aleyna** and **Elif** collaborated on the CASE tools to draw the Subsystem Decomposition and Deployment diagrams, ensuring they followed the non-tier partition approach requested. Simultaneously, **Sinem** and **Esra** drafted the technical rationales for the hardware/software mapping, explaining the choice of MQTT for IoT sensors and HTTPS for mobile applications. **Naciye Nur** developed the Access Control Matrix, categorizing permissions for Managers, Staff, and Citizens. **Elif Sude** finalized the detailed descriptions for the design trade-off table.

Date:	22.12.2025
Location:	Engineering Faculty
Duration:	3 hours
Participants:	Aleyna Gültekin, Esra Sepik, Elif Sude Yılmaz, Elif Özkanat, Naciye Nur Akkuş, Sinem Tanrıver

Content of the meeting (briefly explain the agenda, decisions, work distributions, etc.)

The final meeting was dedicated to the consolidation and quality control of the SDD report. The team reviewed the "Comments on Reports" document to ensure every "red flag" from the previous phase was addressed. **Naciye Nur** coordinated the final formatting of the PDF, ensuring consistency in font, auto-numbering of figures, and the table of contents. **Esra** and **Sinem** reviewed the Subsystem Services section to ensure the lollipop notation was used correctly. **Elif Sude** performed a final check on the traceability of design goals to the functional requirements. The meeting concluded with a collective review of the Boundary Conditions (Start-up, Shutdown, and Failure modes) and the approval of the final work distribution text.